TinyML Keyword Spotting Project

Training

The NN was trained in Python using Google Colab and TensorFlow. The MCU also uses TF Lite Micro for computing the NN in real-time.

Notes on Silence Handling

The dataset has limited silence examples, so the model's "silence" class is only **weakly correlated** with actual silence. In practice, improving silence detection would require **data augmentation** and retraining.

For multi-keyword spotting, simply **add more keyword classes** and retrain. For this project, the detected class (keyword, unknown, or silence) is the one with the **highest probability** from the Softmax output.

> With the current setup, the model essentially acts as a **binary classifier**: keyword vs. everything else. The keyword trained on was "YES."

Performance

- Quick training (1 epoch) achieves ~**95% accuracy**.
- With sufficient epochs, training reaches **99.5%, with 98.5% test accuracy**.
- The model is extremely lightweight, with only ~**11,000 parameters**.

Model Architecture

- Uses **2D convolution** over the spectrogram.
- Input: **61 Mel coefficients** over a **1-second audio frame**.
- Currently implemented in **batch-processing mode**, but can be adapted for **real-time streaming**.
- **CMSIS-DSP** library is used for fast matrix operations and FFT.
- Buffering is implemented in **sliding mode** (not circular).

  Batch processing introduces about **1 second delay**. Streaming can reduce latency to ~**16 ms**.

- FFT parameters:
  - **FFT size:** 512 (32 ms)
  - **Hop size:** 256 (16 ms) → 50% overlap

Core Algorithm: Log-Mel Feature Extraction

The main computational work is generating **log-mel coefficients**:

```
arm_mult_f32(audioFrame, hannWindow, windowed, FRAME_SIZE);
// Apply Hann window

arm_rfft_fast_f32(&fft_inst, windowed, fftOut, 0);
// FFT

arm_cmplx_mag_squared_f32(fftOut, mag, SPECTRUM_BINS);
// Magnitude squared

// Matrix-vector multiply to get Mel bin energies
arm_mat_mult_f32(&melMat, &magVec, &melOut);

// Log + quantization
for(int m = 0; m < NUM_MEL; m++) {
    float log_mel = logf(melRowTmp[m] + 1e-6f);
    int32_t rounded = (int32_t)roundf((log_mel / input_scale) + (float)input_zero_point);
    quantized_mel_row[m] = (int8_t)constrain(rounded, -128, 127);
}
```

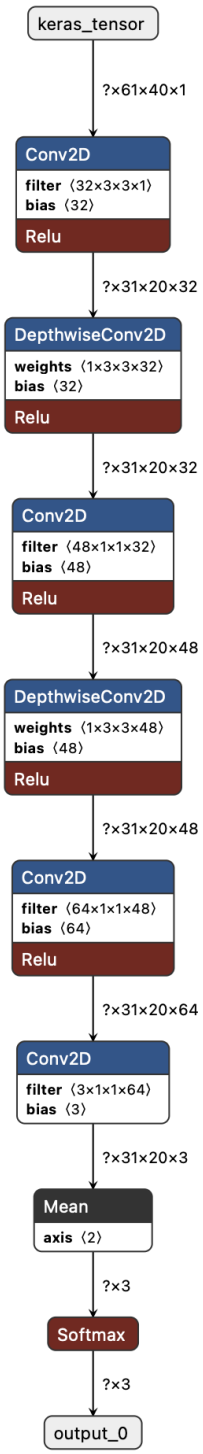

Figure 1: Alt text